Практическая работа 7. Структурный рефакторинг

«**Рефакторинг** — процесс изменения системы ПО таким способом, что при этом не изменяется внешнее поведение кода, но при этом все же улучша-ется его внутренняя структура» [30]. Рефакторинг — нечто вроде чистки кода после того, как он был написан. Пока его цели — решение потенциальных проблем в проекте. Фаулер называет эти проблемные области «дурно пахнущими в коде».

Рефакторинг очень хорошо сочетается с **быстрой разработкой** [66]. Он может проводиться в любом месте итерации, но наиболее эффективно выполнять его либо в конце текущей итерации, либо в начале следующей. Рефакторинг может существенно улучшить внутреннюю структуру кода без изменения его внешнего поведения.

Можно сказать, что практика управляемой тестированием разработки является частичной заменой рефакторинга. Действительно, управляемая тестированием разработка использует разновидность рефакторинга — разновидность, которая применяется для улучшения скорее самого проекта, а не кода. Управляемая тестированием разработка — итеративный и пошаговый процесс, объ единенный с написанием прикладного кода. Рефакторинг может предугадать «дурно пахнущий код» и устранить его до того, как это случится.

7.1. Цели рефакторинга

Фаулер [30] определяет и называет весь диапазон «дурно пахнущего кода». Это — один вид **целей рефакторинга**. Другие цели — не столько результат «дурного запаха», сколько желание далее улучшить код, в особенности, улучшить его структуру [31].

Несколько наиболее интересных целей рефакторинга, вызванных «дурным запахом», следующие:

- дублированный код — одни и те же части кода в нескольких местах;
- длинная подпрограмма-метод — метод, который делает слишком много;
- большой класс — класс, который делает слишком много и/или имеет слишком много элементов данных;
- длинный список параметров — слишком большое количество данных передается в качестве параметров (вместо того, чтобы запросить данные от других объектов);
- расширяющееся изменение — когда класс должен быть изменен в результате больше чем одного вида изменения;
- эффект дробовика — когда одно изменение воздействует на несколько классов;
- излишняя зависимость — метод, который обращается ко многим другим объектам с помощью сообщений `get` (получить), чтобы получить данные для собственных вычислений;
- группы данных — данные (элементы данных, параметры), которые обычно используются вместе во многих местах и которые следует преобразовать в объект.

Цель рефакторинга имеет множество *методов и паттернов рефакторинга*, чтобы устранить «дурные запахи» или внести структурные усовершенствования. Иногда один и тот же метод или паттерн рефакторинга может использоваться, чтобы решать (или частично решать) больше чем одну цель рефакторинга.

7.2. Методы рефакторинга

В жизненном цикле ПО время и трудозатраты на сопровождение кода существенно превышают время и трудозатраты на первоначальное написание кода. При сопровождении кода необходимо прочитать и попробовать понять его, чтобы затем изменить или расширить. Любой рефакторинг кода в процессе его создания, независимо от того, сколь он мал, может принести существенную пользу эксплуатационному персоналу ПО.

Методы рефакторинга (или просто **рефакторинги**) — основные принципы и лучшая практика изменения кода с целью улучшения его понятности, удобства сопровождения и масштабируемости. Изменения кода небольшие, по одному шагу, но усовершенствования могут быть весьма впечатляющими. К сожалению, можно также и ухудшить код, если рефакторинги сделаны не должным образом.

В современной практике CASE- и инструментальные средства разработки программ могут существенно помочь в реализации рефакторинга. Многие инструментальные средства содержат каталоги поддерживаемых рефакторингов. Фаулер [30] является основным источником ссылок, который перечисляет и документирует более шестидесяти методов рефакторинга. Следующее обсуждение иллюстрирует использование методов рефакторинга, рассматривая только три из них:

- *Класс извлечения (Extract Class);*
- *Метод подключения (Subsume Method);*
- *Интерфейс извлечения (Extract Interface).*

7.2.1. Класс извлечения

Одна из целей использования рефакторинга называется *большим классом* — классом, который делает слишком много и/или имеет слишком много элементов данных. Большие классы могут возникнуть из-за чрезмерной минимизации связей между классами. В случае большого класса в коде итерации 1 уместны два метода: *Класс извлечения* (Extract Class) и *Интерфейс извлечения* (Extract Interface) [30].

Рефакторинг Класс извлечения определяется так: «Создайте новый класс и переместите соответствующие поля и методы из старого класса в новый» [30]. Главная трудность заключается в определении, как разбить большой класс на ряд меньших классов. Идея заключается в том, чтобы извлечь непротиворечивые и объединенные части функциональных возможностей в отдельный класс (классы).

Как только поля и методы будут перемещены в новый класс (классы), должна быть установлена связь ассоциации от старого класса к новому. Это может быть односторонняя связь, если нет очевидной потребности в обратной передаче сообщений. Конечно, перемещение методов должно влиять на классы клиента, которые зависели от этих методов. Это требует изменений в обращениях к методам в этих классах.

Рис. 1 показывает, как рефакторинг *Класс извлечения* мог бы быть ис-пользован для класса `CActioner` — класс «исполнитель» пакета `control`

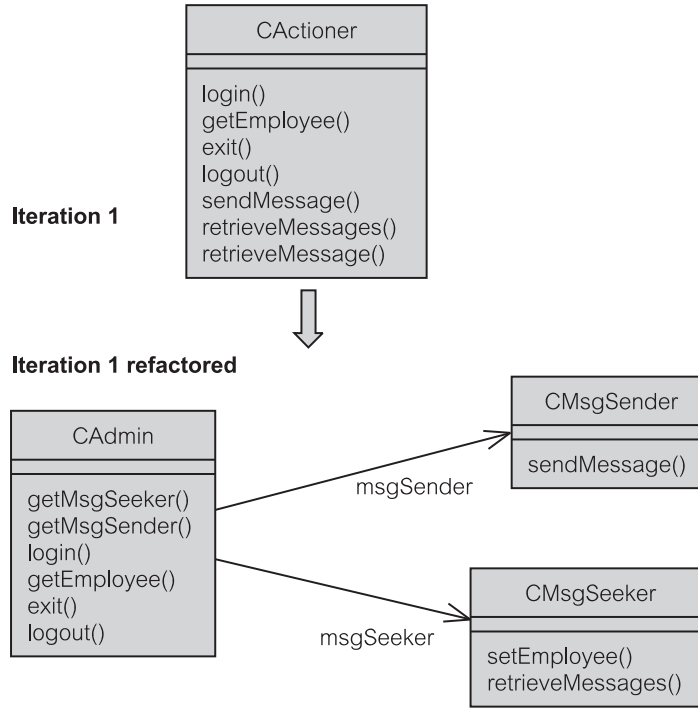


Рис. 1. Рефакторинг Класс извлечения

Класс `CActioner` включен в две совершенно несопоставимые задачи: извлечение исходящих сообщений, требуемых пользователем, и посыл-ка исходящих сообщений (передача по электронной почте). Логично извлечь эти две задачи в отдельные классы: `CMsgSeeker` (класс «поиск сообщения» пакета `control`) и `CMsgSeeker` (класс «передача сообщения» пакета `control`). Чтобы избежать терминологического беспорядка, `CActioner` переименован в `CAdmin` (класс «администратор» пакета `control`). Конструктор и методы, не являющиеся общедоступными, не рассматриваются.

Как ожидается, с помощью рефакторинга *Класс извлечения* `CAdmin` обеспечивает связи ассоциации с новыми классами. Связи поддерживаются двумя методами: `getMsgSeeker()` (получение поиска сообщения) и `GetMsgSender()` (получение передачи сообщения). Первый получает объект `CMsgSeeker`, который отвечает за извлечение исходящих сообщений для уровня `presentation` (представление). Второй получает объект `CMsgSender`, ответственный за исходящие сообщения, передаваемые по электронной почте.

Рефакторинг дает один новый метод (`setEmployee()` — задать информацию о служащем). Метод `setEmployee()` используется, чтобы установить ассоциацию от `CMsgSeeker` к `EEmployee` (класс «служащий» пакета `entity`). Это требуется, потому что извлечение исходящего сообщения требует данных о служащем. Метод `setEmployee()` получает объект `EEmployee` от метода `login()` (регистрационное имя) класса `CAdmin()`.

7.2.2. Метод подключения

Дублированный код (те же самые части кода в нескольких местах) является частой целью рефакторинга. В зависимости от своей природы к дублированному коду можно подойти различными способами. Типичный рефакторинг для дублированного кода — *Метод извлечения*, то есть, преобразование кода, дублированного в нескольких методах, в отдельный метод [30]. Другой подходящий рефакторинг, не перечисленый Фаулером, — Метод подключения (Subsume Method). **Рефакторинг Метод подключения** устраняет метод включением его функциональных возможностей в другой существующий метод.

Рефакторинг *Класс извлечения* на рис. 1 показывает, что метод `retrieveMessage()` (извлечение сообщения) класса `CActioner` (класс «исполнитель» пакета `control`) отсутствует в улучшенном коде. Метод `retrieveMessage()` удален не рефакторингом *Класс извлечения*. Это реализуется *Методом подключения*, который объединяет методы `retrieveMessage()` в метод `retrieveMessages()` (извлечение сообщений).

Неудачно, что код итерации 1 может быть запущен двумя разными опциями меню, доступными пользователю: *View Unsent Messages* (просмотр непосланных сообщений) и *Display Text of a Message* (отображение текста сообщения) — рис. 12.12. Первая опция приводит к извлечению и отображению нескольких исходящих сообщений; вторая — к извлечению и отображению одного исходящего сообщения, которое пользователь предназначает для передачи по электронной почте. В обоих случаях извлекается полное содержание данных объектов `EOutMessage` (класс «исходящее сообщение» пакета `entity`), но отображение информации пользователю изменяется.

Суть проблемы в том, что имеется большое количество дублированного кода, вытекающее из независимой обработки этих двух действий меню. Конечно, извлекая несколько исходящих сообщений и извлекая единственное исходящее сообщение, используется схожая обработ-ка. Рис. 2 демонстрирует две последовательности передачи сообщений для итерации 1 и то, как они объединены в единственный путь, начинающийся с уровня `control` (управление). Различие между несколькими исходящими сообщениями и единственным исходящим сообщением очевидно только на уровне `presentation` (представление).

7.2.3. Интерфейс извлечения

Цель рефакторинга Интерфейс извлечения двойная и определяется так: «Несколько клиентов используют то же самое подмножество интерфейса класса или два класса содержат общую часть своих интерфейсов» [30]. Метод **рефакторинга Интерфейс извлечения** используется, чтобы «выделить подмножество в интерфейс» (там же). Идея относительно этого рефакторинга связана с самой природой интерфейсов.

Подобно *Классу извлечения Интерфейс извлечения* часто является ответом на цель, связанную с большим классом. Это — периодичес-ки повторяющаяся цель рефакторинга в итерации 1. Класс `PConsole` (класс

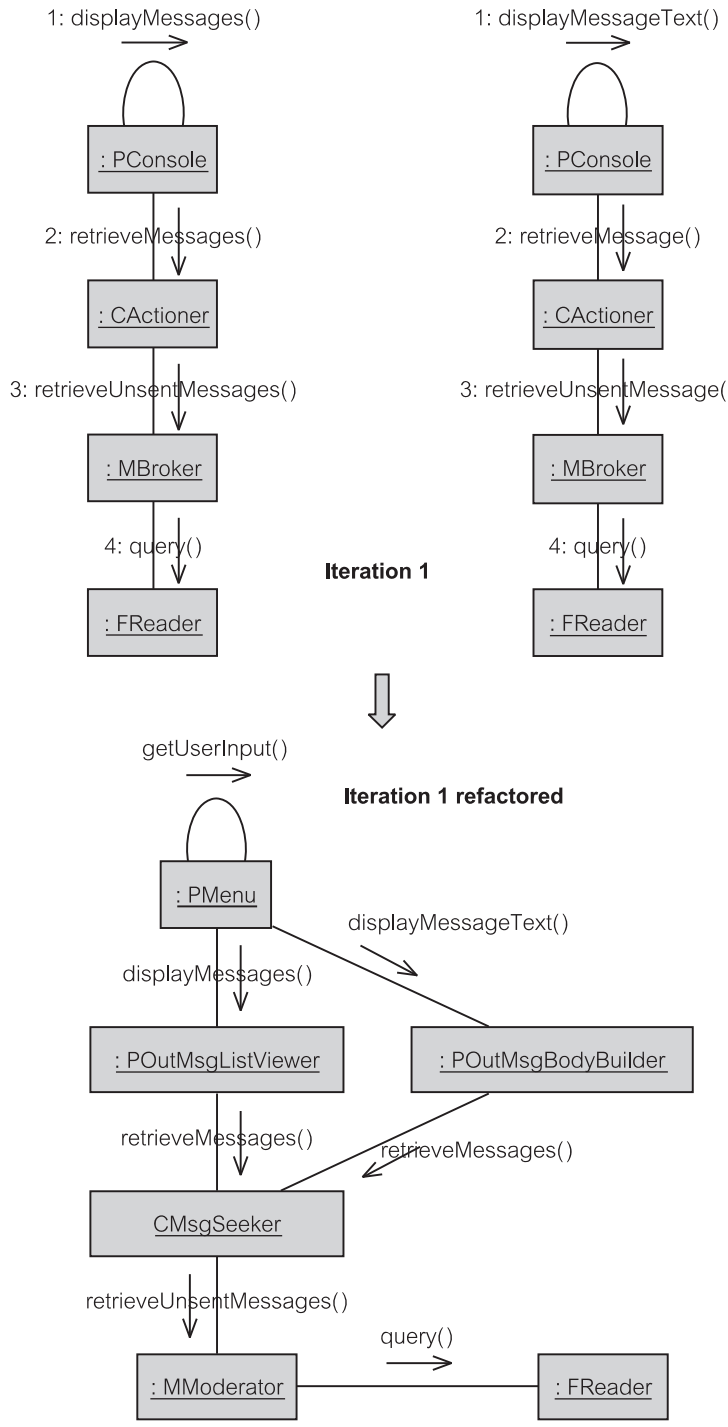


Рис. 2. Рефакторинг Метод подключения

«консоль» пакета presentation) — большой класс, который делает слишком много. В зависимости от команды меню, выбранной пользователем, PConsole исполняет различные задачи. Например, может потребоваться показать список исходящих сообщений или детали отдельного исходящего сообщения.

В итерации 1 выполнение этих различных задач инкапсулировано методом displayMenu() (отобразить меню) — листинг 13.16. Этот ме-тод содержит приватный (private) метод getUserInput() (получить введен-ную пользователем информацию), который получает запрос пользователя и останавливается на задаче, вызывая соответствующий метод PConsole, наподобие displayMessages() (отобразить сообщения). Класс PConsole имеет только два общедоступных (public) метода: displayLogin() (отобразить регистрационное имя) и displayMenu(). Все остальное инкапсулировано (скрыто). Инкапсуляция имеет преимущества, но вызывает «запахи» большого класса, которые нелегко обслуживать и устранить.

Рефакторинг класса PConsole может сначала использовать метод *Класс извлечения*, чтобы создать, например, два отдельных класса для отображения списка исходящих сообщений и отдельного исходящего сообщения (рис. 3). Соответствующие обязанности от PConsole (переименованного теперь в PMenu — класс «меню» пакета presentation) берут на себя два новых класса POutMsgListViewer (класс «просмотр списка исходящих сообщений» пакета presentation) и POutMsgBodyBuilder (класс «формирование тела исходящего сообщения» пакета presentation).

Естественный следующий шаг должен подтвердить, что два новых класса уровня presentation (представление) в улучшенном коде используют один и тот же метод display() (отобразить), чтобы показать строку на экране. Ясно, что код улучшит удобочитаемость и документирование, если метод display() будет извлечен в интерфейс, реализован в PMenu и использован при необходимости новыми классами уровня presentation.

Рис. 3 представляет улучшенный проект, используя *Класс извлечения* и *Интерфейс извлечения* на уровне presentation. Рефакторинг этого уровня для итерации 2 имеет ограниченный объ ем. Переход в итерации 2 к графическому пользовательскому интерфейсу означает весьма радикальную реорганизацию уровня presentation. Даже в рассматриваемом случае наличие интерфейса и новых классов облегчит этот переход. В некоторых случаях был бы оправдан более радикальный рефакторинг. Например, класс POutMsgBodyBuilder может быть разбит на два класса, чтобы отделить отображение исходящего сообщения от подготовки его для передачи по электронной почте.

7.3. Паттерны рефакторинга

Рефакторинг неотделим от структурного проектирования и разработки шаблонов. Существенные структурные усовершенствования могут быть достигнуты скоординированным применением многих методов рефакторинга. Начальное проектирование системы управляется структурными паттернами. Структурные паттерны также полезны в рефакторингах, чтобы получить более существенные исправления или расширения кода.

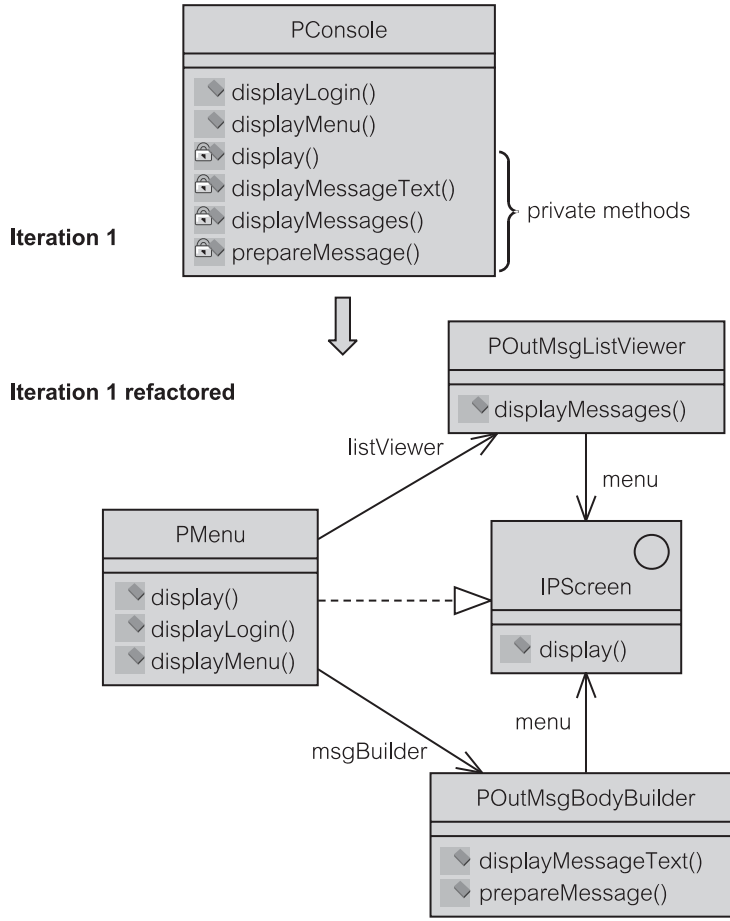


Рис. 3. Рефакторинг Интерфейс извлечения

Паттерны рефакторинга — структурные паттерны, используемые в рефакторинге кода. Фаулер [31] называет их паттернами структуры промышленного приложения. Они являются целью тех аспектов системы, которые делают ее промышленным приложением в противоположность маломасштабным настольным приложениям. Они имеют дело с такими проблемами, как связь с БД, кэши сохраняемых объектов, расположенные в памяти, управление транзакциями и параллелизмом, представление Web-технологий, работа с распределенными объектами и т. д.

Паттерны рефакторинга обеспечивают системе более легкую масштабируемость, когда она перерастает в крупномасштабное решение. Итерация 1 учебного примера EM подчеркивает присутствие БД в приложении при сохранении весьма примитивного текстового пользовательского интерфейса. Итерация 2 делает шаг к графическому пользовательскому интерфейсу. Паттерны рефакторинга, рассматриваемые в данной главе, все еще применяются

к итерации 1 и поэтому концентрируются на проблемах, отличных от проблем пользовательского интерфейса.

Следующее рассмотрение иллюстрирует структурные паттерны, используемые для рефакторинга кода итерации 1 в пределах уровня `domain` (предметная область). Большинство этих паттернов описано у Фаулера [31]. Книга Фаулера является источником ссылок, находящимся на современном уровне, и которая перечисляет и документирует более пятидесяти структурных паттернов. Паттерны представлены следующими группами:

- *Коллекция идентичности объектов* (Identity Map);
- *Преобразователь данных* (Data Mapper);
- *Загрузка по требованию* (Lazy Load);
- *Единица работы* (Unit of Work).[1]

7.3.1. Коллекция идентичности объектов

Итерация 1 подтверждает важность назначения идентификаторов объектов (object identifier — OID) объектам пакета `entity` — сущность и важность поддержания кэшей пакета `entity` с помощью пакета `mediator`— посредник (раздел 13.6). Это, конечно, хорошо, но имеющийся в настоящее время класс `MBroker` (класс «посредник» пакета `mediator`) «пахнет» большим классом, поэтому можно использовать ряд *паттернов рефакторинга*, чтобы улучшить управление идентификацией и кэшированием. Один такой паттерн — **Коллекция идентичности объектов** (Identity Map) [31].

Паттерн *Коллекция идентичности объектов* «гарантирует, что каждый объект загружается только однажды, храня ссылку на каждый загруженный объект в коллекции. При использовании объектов просматривается коллекция, содержащая ссылки на них» [31]. Объект *Коллекция идентичности объектов* поддерживает одну или несколько коллекций (например, хэш-коллекции), которые содержат идентификаторы объектов. *Коллекция идентичности объектов* имеет два основных назначения:

- гарантировать, что одна и та же запись БД не загружена несколько раз в различные объекты приложений (это создало бы хаос, если приложение изменило бы один из этих объектов, а другие — нет);
- избежать ненужной загрузки одних и тех же данных несколько раз (это воздействовало бы на выполнение функций приложения).

Фаулер [31] различает явно заданные и общие коллекции идентичности объектов. К объектам в *явно заданной коллекции идентичности объектов* можно иметь доступ (`get` — получить), их можно зарегистрировать (`put` — поместить) и снять с них регистрацию (`remove` — удалить) с использованием различных методов для каждого класса кэшированных объектов. Например, для получения доступа к объекту:

```
getEEmployee (new Integer(empOID));
```

[1] Здесь также можно отметить, что русская терминология паттернов еще не устоялась, и порой используются названия, не соответствующие английским названиям. Так, например, паттерн Identity Map часто называют Коллекцией объектов, что, на наш взгляд, искажает его суть. Мы выбрали здесь наиболее подходящую по нашему мнению терминологию. (*Прим. перев.*)

Для объектов в *общей коллекции идентичности объектов* доступ (`get`), регистрация (`put`) и снятие регистрации (`remove`) выполняются единственным методом для всех классов. Параметр метода определяет класс. Например, для получения доступа к объекту:

```
get ("EEmployee", new Integer(OID));
```

Общая коллекция может использоваться только в тех случаях, если все идентификаторы объекта формируются одним и тем же алгоритмом и глобально уникальны для всех объектов всех классов. Общая коллекция позволяет легко создавать и поддерживать единую регистрацию идентификаторов объектов, так что другие объекты могут использовать ее, чтобы найти объект по его OID.

Общая коллекция имеет преимущество удобочитаемости и документирования. Все доступные коллекции статически объявлены в коде. Дополнения и удаления в коллекциях ограничены единственным классом, так что добавление и удаление новых методов не является проблемой.

Далее Фаулер [31] различает одну коллекцию на класс и одну коллекцию на сеанс. *Одна коллекция на сеанс* требует глобальных уникальных идентификаторов объектов. В этом смысле она идет рука об руку с общей коллекцией идентификации объектов. *Одна коллекция на класс* может использовать идентификаторы объектов, уникальные в пределах класса, но здесь возникает проблема, как управлять классами в дереве наследования. В обоих случаях для коллекций идентичности обновляемых объектов должна быть обеспечена защита в виде транзакций и они должны быть помечены как измененные, когда объекты идентичности выходят из синхронизации с соответствующими записями БД.

В случае итерации 1 результатом паттерна *Коллекция идентичности объектов*, так же как и метода *Класс извлечения*, является рефакторинг, который приводит к новому классу `EIdentityMap` — класс «коллекция идентичности объектов» пакета `entity` (листинг 15.1). `EIdentityMap` определяет два вида коллекций (строки 19–20). Одни коллекции содержат `OID` пакета `entity`, а другие коллекции содержат `msgID` (идентификатор объектов-сообщений) к `OID EOutMessage` (класс «исходящее сообщение» пакета `entity`). Итерация 1 не требует других коллекций, но они могут быть добавлены всякий раз, когда это будет необходимо. Пример показывает методы, необходимые, чтобы управлять размещением объектов `EContact` (класс «деловой партнер» пакета `entity`).

Листинг 15.1. Класс EIdentityMap — фрагмент

Класс EIdentityMap — фрагмент

```
18:  public class EIdentityMap {
21:     private Map OIDToObj; //OID -> Obj
22:     private Map msgPKToOID; //msgPK -> OID
23:
24:     public EIdentityMap() {
25:       OIDToObj = new HashMap();
```

```
28:       msgPKToOID = new HashMap();
29:     }
30:
31:     /** Получить хранимого делового партнера */
32:     public IAContact findContact(int contactOID) {
33:       return (IAContact) OIDToObj.get(new
                                    Integer(contactOID));
34:     }
35:     /** Разместить делового партнера с заданным OID */
36:     public void registerContact(IEObjectID oidObject) {
37:       OIDToObj.put(new Integer(oidObject.getOID()),
                                                 oidObject);
41:     }
42:     /** Снять регистрацию с зарегистрированного делового
                                          партнера */
43:     public void unregisterContact(IEObjectID oidContact) {
44:       OIDToObj.remove(new Integer(oidContact.getOID()));
46:     }
113: }
```

`EIdentityMap` — *явно заданная коллекция идентичности объектов* — она имеет отдельные методы для каждого класса пакета `entity`. Класс представляет стратегию *одной коллекции на сеанс* — он использует глобальные уникальные идентификаторы `OID`.

7.3.2. Преобразователь данных

Паттерн Преобразователь данных (Data Mapper) определяется как «уровень *Преобразователей* (Mappers), который перемещает данные между объектами и БД, когда они хранятся независимо друг от друга, и непосредственно сам преобразователь» [31]. Общий паттерн *Преобразователь*, упомянутый в определении, является «объектом, который устанавливает связь между двумя независимыми объектами» [31]. Паттерн *Преобразователь* отличается от паттерна *Посредник* в том, что объ екты,выделенные объек-том-преобразователем, не ведают друг о друге, в то время как объ екты, ис-пользующие посредника, знают об этом.

В PCMEF-шаблоне *Преобразователь данных* принадлежит пакету `mediator` (посредник). В итерации 1 класс `MBroker` — класс «посредник» па-кета `mediator` отвечает, среди прочего, за знание объектов па-кета `entity` (сущность), расположенных в кэше, и за требование извлечения данных из БД, если объект не находится в кэше или если кэш изменен. Эти обя-занности *Преобразователя данных* необходимо извлечь из `MBroker` в отдель- ный класс (классы), используя рефакторинг *Класс извлечения*. `MDataMapper` (класс «преобразователь данных» пакета `mediator`) — один такой новый класс в улучшенном коде итерации 1.

Рис. 4 представляет UML-**диаграмму деятельности**, ко-торая показывает, как `MDataMapper` отделяет пакет `entity` (ответствен-ный за кэш) от пакета `foundation` — основание (ответственного за доступ

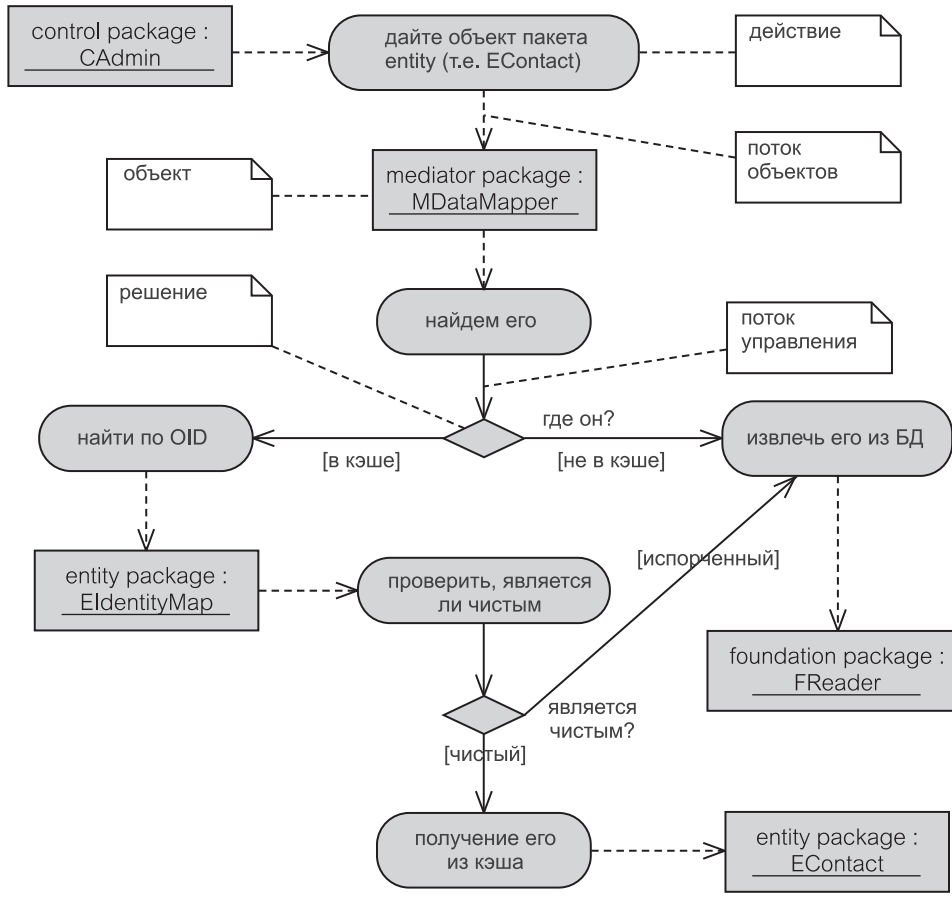


Рис. 4. Диаграмма деятельности для MDataMapper

к БД). Диаграмма деятельности используется здесь скорее для удобства, нежели как демонстрация ее обычной применимости.

Семантике модели действий на рис. 4 легко следовать. Значение графических объ ектов объ ясняется в примечаниях UML. Прежде всего объект управления (`CAdmin` — класс «администратор» пакета `control`) запрашивает `MDataMapper`, чтобы тот получил объект пакета `entity`. `CAdmin` может выполнить этот запрос рядом способов:

- `CAdmin` может знать OID объекта и передать его объекту `MDataMapper` или
- `CAdmin` может знать значения некоторых атрибутов объекта, например, значение первичного ключа, и передать эту информацию объекту `MDataMapper` как условие поиска или
- `CAdmin` может запросить ссылочный поиск, обратившись к `MDataMapper`, чтобы он нашел объекты, которые связаны с объектом, известным объекту `CAdmin`.

MDataMapper должен установить, где находится объект (объекты). Находится ли он в кэше памяти, или он должен быть извлечен из БД? Если он находится в кэше памяти, то является ли он измененным? Если да, то он должен быть извлечен из БД. Объект EIdentityMap — класс «коллекция идентичности объ ектов» пакета entity (листинг 15.1) знает, находится ли нужный объ ект в кэше. Если EIdentityMap содержит объекты, и те не изменены, они будут возвращены объ екту MDataMapper и затем объекту CAdmin. Если же нет, MDataMapper формирует соответствующие SQL-запросы и передает их объ екту FReader (класс «чтение» пакета foundation) для доступа к БД.

Загрузка — импорт

Рис. 4 не содержит полный набор операций MDataMapper. Когда записи данных будут извлечены их БД, MDataMapper превращает их в объекты (то есть, MDataMapper создает новые объ екты пакета entity). Затем новые объекты регистрируются в EIdentityMap. Это — операция **загрузки** (записи БД извлекаются и преобразуются в объ екты памяти). С точки зрения БД загрузка является операцией **импорта из БД — check-out** (записи импортируются в программу). Процесс загрузки также называется **материализацией** [55].

Фрагмент диаграммы последовательности действий на рис. 5 иллюстрирует возможный сценарий для загрузки объ екта в улучшенном коде итерации 1. Диаграмма касается взаимодействия «Регистрационное имя». Она показывает, как MDataMapper извлекает и создает новый объ-ект EEmployee (класс «служащий» пакета entity), когда его запрашивают,

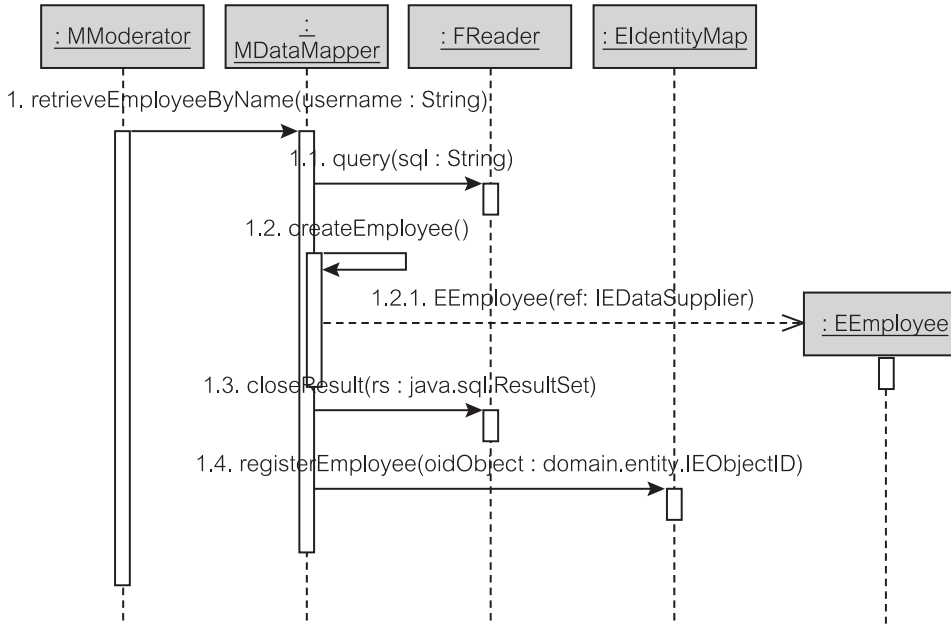


Рис. 5. Диаграмма последовательности для загрузки объекта

чтобы получить имя служащего. Эта операция включает формирование OID для EEmployee, который затем передается объекту EIdentityMap в параметре метода registerEmployee() (регистрация служащего).

Выгрузка — экспорт

Вполне естественно, что если *Преобразователь данных* ответствен за загрузку объектов из БД, он должен также обеспечивать и противоположную операцию — выгрузку объектов в БД. С точки зрения БД это операция **экспорта в БД — check-in**. Процесс **выгрузки** также известен как **пассивация** или **дематериализация** [55]. Операция выгрузки требуется, когда:

- приложением создан новый объект пакета entity, и необходимо его постоянно хранить в БД или
- объект пакета entity обновляется приложением, и изменения должны постоянно храниться в БД или
- объект пакета entity удален приложением и поэтому соответствующая запись должна быть удалена и из БД.

Выгрузка является темой исследования бизнес-транзакций, а также операций commit (фиксация) и rollback (откат).

Итерация 1 не создает и не удаляет объекты пакета entity, но она дол-жна корректировать объекты EOutMessage (класс «исходящее сообщение» пакета entity), чтобы указать, что они были переданы по электронной по-чте. Это — задача взаимодействия «Сообщение, передаваемое по электрон-ной почте». Фрагмент диаграммы последовательности дей-ствий на рис. 6 иллюстрирует, как «скорректированная» выгрузка может быть реализована в улучшенном коде итерации 1.

CMsgSeeker (класс «поиск сообщения» пакета control) посылает исходящее сообщение и затем обращается к MModerator (класс «координатор» пакета mediator), чтобы тот выполнил метод updateMessage() (скорректировать сообщение). MModerator делегирует эту задачу объекту MDataMapper, который создает оператор корректировки SQL. FWriter (класс «запись» пакета foundation) делает изменение в БД. Это разрешает объ екту MDataMapper выполнить метод unregisterMessage() (снять регистрацию сообщения) в объ екте EIdentityMap. Частью этой операции является выполнение метода flagCache() (установить флаг кэша), кото-рый фиксирует, что кэш изменен. Пример предполагает, что MDataMapper имеет собственную коллекцию OID (идентификатор объекта).

7.3.3. Альтернативные стратегии Преобразователя данных

В улучшенной итерации 1 используется единственный класс *Преобразователя данных* (MDataMapper — класс «преобразователь данных» пакета mediator), чтобы размещать и удалять строки данных у всех объектов пакета entity (сущность), известных в приложении (EEmployee — класс «служащий» пакета entity, EContact — класс «деловой партнер» пакета entity и EOutMessage — класс «исходящее сообщение» пакета entity).

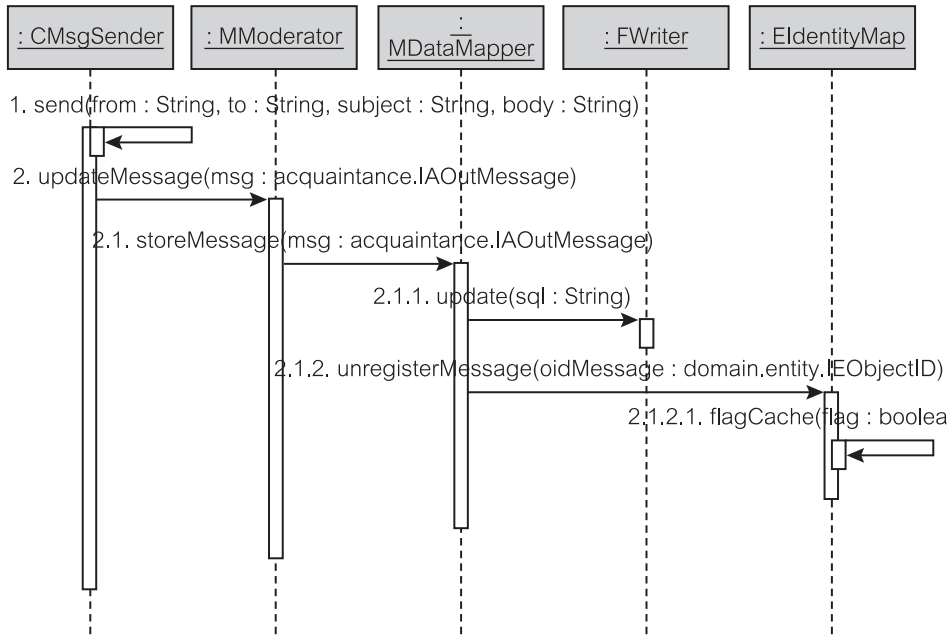


Рис. 6. Диаграмма последовательности для операции экспорта класса MDataMapper

`MModerator` (класс «координатор» пакета `mediator`) служит Фасадом (см. 9.3.1) для пакета `mediator` (посредник). `MDataMapper` обеспечивает ассоциации к `EIdentityMap` (класс «коллекция идентичности объектов» пакета `entity`), с одной стороны, и к `FReader` (класс «чтение» пакета `foundation`) и `FWriter` (класс «запись» пакета `foundation`), с другой стороны. Это показано на рис. 7.

Более сложные системы или последовательные итерации могут требовать альтернативных решений, основанных на различных паттернах рефакторинга. Следующий список точно определяет общие альтернативные стратегии [55, 31]:

- Несколько *Преобразователей данных*, возможно, по одному на каждый класс пакета `entity` (сущность).
- Использование метаданных и Java-отображения, чтобы динамически формировать в приложении *Преобразователи данных* по мере необходимости.
- Экспорт по требованию, который размещает только данные, необходимые в настоящее время приложению, и, если возможно, создает пустые объекты (модули) для данных, не извлеченных из БД, но которые связаны с уже извлеченными данными.

Несколько Преобразователей данных

`MDataMapper` в улучшенной итерации 1 имеет отдельные методы для управления запросами клиента, направленные на различные объекты пакета `entity`. Это необходимо, потому что имеется много различных способов,

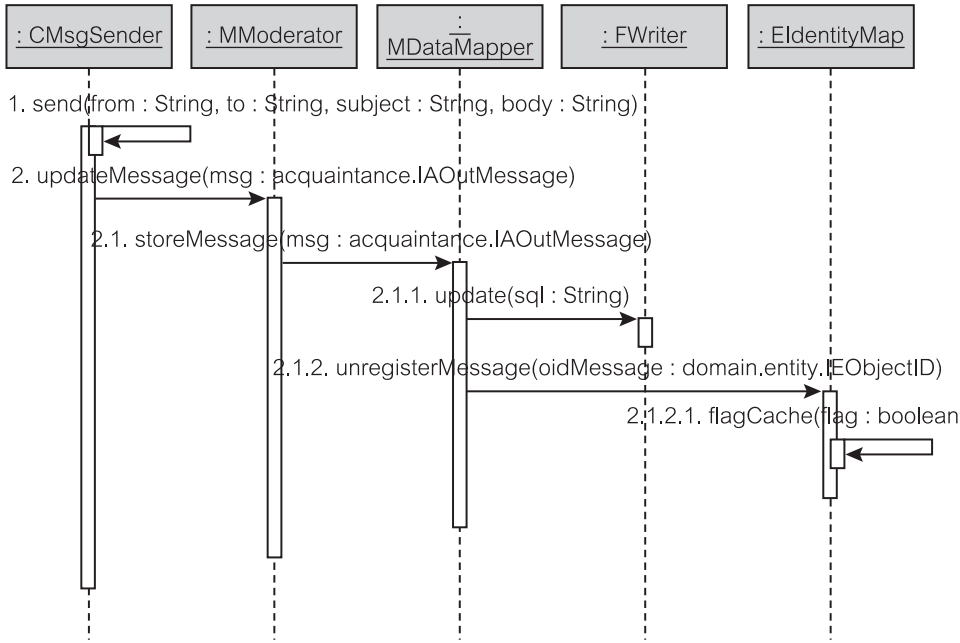


Рис. 7. MDataMapper в улучшенной итерации 1

которыми *Преобразователь данных* запрашивает объ екты пакета `entity`. Кроме того, каждый объект пакета `entity` имеет свои собственные уникальные элементы данных и ассоциации к другим объ ектам. В случае большого числа классов пакета `entity`, управляемых приложением, единственный *Преобразователь данных* может стать слишком разросшимся классом.

Рис. 8 показывает альтернативное решение итерации 1 с несколькими преобразователями данных — один *Преобразователь данных* на каждый класс пакета `entity`. Три преобразователя реализуют, каждый уникальным способом, интерфейс `IMDataMapper` (интерфейс «преобразователь данных» пакета `mediator`). `MModerator` обеспечивает ассоциацию «один ко многим» к объ ектам `IMDataMapper` [55].

Назначение методов следующее:

- `getByOID()` — получение объекта пакета `entity`, если дан его `OID` (идентификатор объекта); извлечение его из БД, если кэш изменен.
- `retrieve()` — извлечение объекта пакета `entity` из кэша (если он там) или из БД; если имеет место второй случай, создание нового объекта пакета `entity` и размещение его в кэше.
- `insert()` — преобразование нового объекта пакета `entity` в строку данных и размещение его в БД.
- `update()` — сохранение изменений объекта пакета `entity` в БД.
- `delete()` — удаление объекта пакета `entity` из БД и устранение его из кэша.

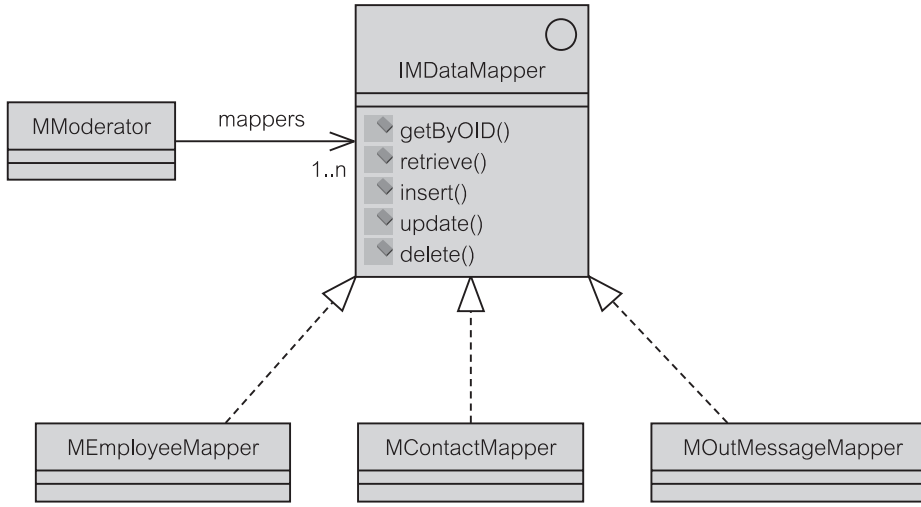


Рис. 8. Несколько Преобразователей данных

Обратите внимание, что если для каждого класса пакета `entity` используется отдельный *Преобразователь данных*, то может быть также желательно иметь одну *Коллекцию идентичности объектов* на каждый класс пакета `entity`. Тогда каждый преобразователь будет обслуживать свой собственный кэш.

Преобразование метаданных

Цель **паттерна Преобразователь метаданных** [31] — динамически формировать объектно-реляционное преобразование, основанное на **метаданных** (данные относительно данных). Метаданные хранят знания относительно преобразований таблиц и классов, столбцов и элементов данных, внешних ключей и связей ассоциации и т. д.

Программа может опрашивать метаданные во время выполнения и динамически строить SQL-операторы, с одной стороны, и реализовывать получение/задание объектов пакета `entity`, с другой стороны. С точки зрения масштабируемости *Преобразователь метаданных* позволяет легко адаптировать приложение к изменениям схемы БД и к изменениям, определяемым данными, в частности, к дополнениям классов пакета `entity`.

Важное решение, связанное с *Преобразователем метаданных*, состоит в том, где хранить метаданные. Имеются три возможности [31]:

- непосредственно в *исходном коде* приложения (в пакете `mediator` в случае PCMEF-шаблона);
- во *внешнем файле*, предпочтительно в XML-файле (приложение читает файл во время инициализации и создает соответствующие преобразования в структуре программы);
- в *БД* (это позволяет использовать одни и те же метаданные несколькими приложениями и допускает автоматическую проверку самых последних изменений в схеме БД).

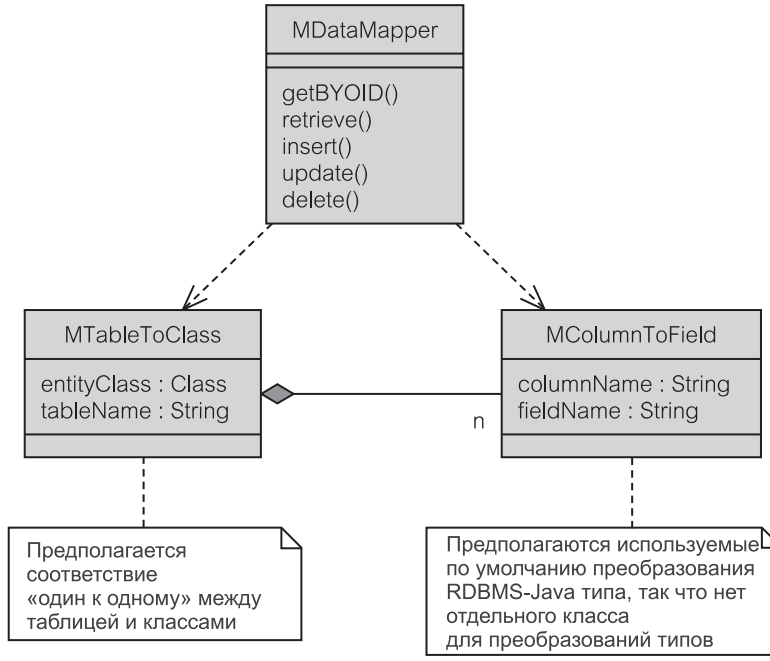


Рис. 9. Паттерн Преобразователь метаданных

Имеются две основные стратегии включения информации размещения в выполняемом коде [31]:

- *формирование кода*, который читает информацию метаданных и формирует классы, чтобы выполнить преобразование (это делается во время процесса формирования кода до компиляции программы);
- *рефлексивная программа*, которая позволяет настраивать приложение к любой новой информации метаданных во время выполнения.

В PCMEF классы, которые могли бы формировать *Преобразователь метаданных*, принадлежат пакету `mediator`. Рис. 9 представляет простую модель классов для *Преобразователя метаданных*. Однако модель не реализована в учебном примере EM.

`MDataMapper` зависит от этих двух классов метаданных для определения информации преобразования, необходимой для выполнения его методов. Методы, которые извлекают информацию из БД, типа `retrieve()`, будут запрашивать `MTableToClass` (класс «таблицу в класс» пакета `mediator`), чтобы получить `entityClass` (класс пакета `entity`), передающий `tableName` (имя таблицы), из которой извлекаются данные. Методы, которые пишут информацию в БД, типа `insert()`, будут запрашивать `MTableToClass`, чтобы получить `tableName` и `MColumnToField` (класс «столбец в поле» пакета `mediator`) для определения `columnNames` (имена столбцов).

Контрольные вопросы:

1. Что понимается под термином «рефакторинг», и какова его главная цель по отношению к внешнему поведению и внутренней структуре ПО?
2. Какие изменения в связях между классами и классах-клиентах необходимо внести после применения рефакторинга Extract Class?
3. В чем состоит назначение метода рефакторинга «Метод подключения» (Subsume Method) и чем он отличается от метода «Метод извлечения» (Extract Method)?
4. Как совместное применение методов Extract Class и Extract Interface на уровне представления (presentation) помогает подготовить систему к переходу от консольного интерфейса к графическому (GUI)?
5. Что представляют собой паттерны рефакторинга (структурные паттерны корпоративных приложений по М. Фаулеру) и какие проблемы они решают?
6. Каковы роли процессов загрузки (материализации / check-out) и выгрузки (дематериализации, пассивации / check-in) в контексте работы Data Mapper?
7. В каких случаях использование единственного универсального класса Data Mapper становится неэффективным и требует разделения на несколько специализированных преобразователей?
8. Перечислите базовые методы типового интерфейса IMDataMapper (getByOID, retrieve, insert, update, delete) и опишите их назначение.
9. В чем заключается цель паттерна «Преобразователь метаданных» и какие преимущества он дает при изменении схемы БД?
10. Назовите три способа хранения метаданных в приложении, а также их достоинства и недостатки.